

The Amari - Hopfield Model

弭元元

清华大学 心理系

The Synaptic Plasticity

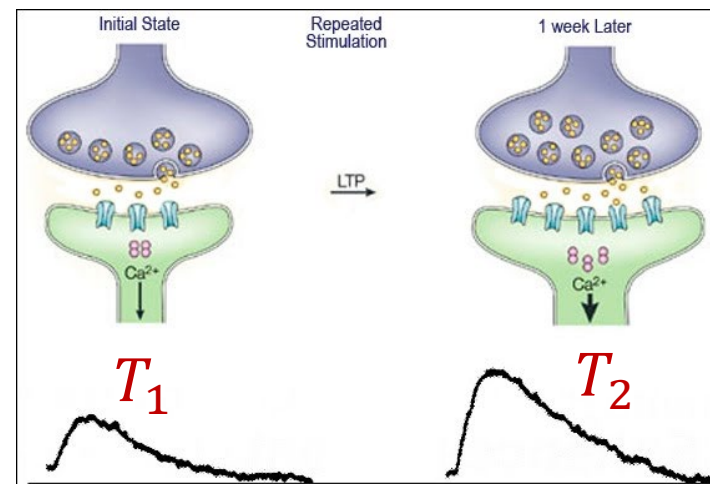
根据突触连接强度变化所能维持的时长，突触可塑性主要被分为两类：

- **短时程可塑性** (Short-Term Plasticity, STP)

出现STP的原因大都是物理层面的可逆变化，如神经递质的消耗，持续时间一般为**毫秒到秒**的量级，是神经系统完成实时计算的核心。

- **长时程可塑性** (Long-Term Plasticity, LTP)

- ✓ LTP的持续时间可以是**几十秒以上，甚至是不可逆**（比如，终生）。这就暗示了突触在生化层面发生了一些反应，最终改变了信号传递的强度。
- ✓ LTP对认知功能有重要影响，如神经科学家普遍认为记忆就存储在神经元的连接中（如Amari-Hopfield Model所采用的策略），而神经系统依靠突触的长时程可塑性来产生或改变记忆。
- ✓ Long term depression (**LTD**) & Long term Potentiation (**LTP**)

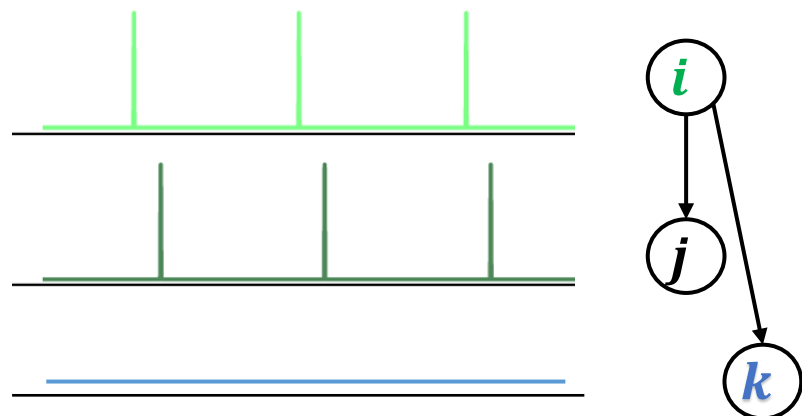


Hebb Learning Rule

Hebb学习规则：描述突触长时程可塑性的经典数学模型，由加拿大心理学家唐纳德·赫布于1949年在《The Organization of Behavior》书中提出，又称为赫布理论。

- 当神经元*i*对神经元*j*产生持续重复的刺激时，*i*和*j*的连接强度会增加；
- Hebb学习规则只关心两个神经元在一段时间内的同步发放，不关心发放的次序

Cells that fire together wire together !



$$x_i = 1 \ \& \ x_j = 1 \ \& \ x_k = -1$$

$$\Delta w_{ji} = \eta, \Delta w_{ki} = -\eta$$

□ 定义神经元的活动状态为 x 。

- 若该神经元处于活跃的状态，则 $x = +1$
- 若该神经元处于静息的状态，则 $x = -1$

□ 定义神经元*i*投射到神经元*j*的突触连接强度 w_{ji}

$$\text{Hebb学习规则: } \Delta w_{ji} = \eta x_j x_i$$

The Amari-Hopfield Model

whose fundamental job is to **store** and **retrieve** information.

--- Amari, 1972; Hopfield, 1982

In the past few years, they experienced a surge of interest due to novel theoretical results pertaining to their **information storage capabilities**, and their **relationship** with SOTA AI architectures, such as Transformers and Diffusion Models.

神经元是大脑的计算单元，而突触则承担信息存储的职责

The Amari-Hopfield Model

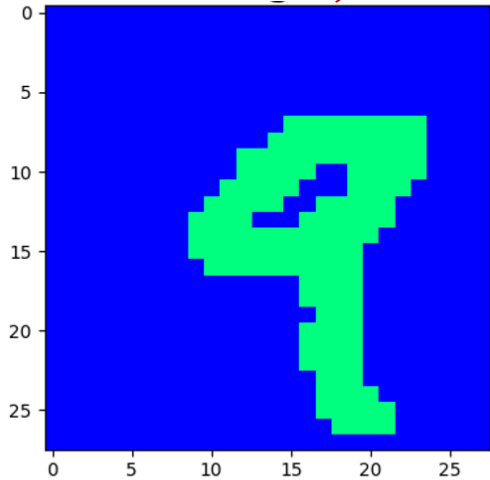
A self-organizing net has the property that its structure gradually varies, depending on the stimulus patterns applied to the net from the outside.

When stimulus patterns or sequences of patterns are repeatedly applied to the net, it is expected that the net **learns from** these stimuli and **fixes** some of them by self-organization as **stable equilibrium states or sequences of state transitions**. It may be said that the net "remembers" these patterns or pattern sequences.

Once a pattern is remembered as a stable equilibrium state, it will be **recalled and reproduced** correctly when a neighboring pattern is given as a stimulus. Similarly, once a pattern sequence is remembered, by giving a pattern in the sequence as a cue stimulus, the part of the sequence that follows the cue pattern is successively recalled.

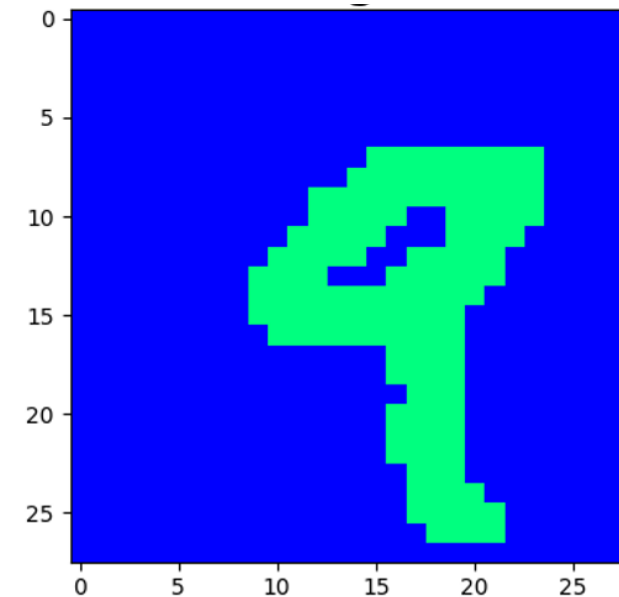
A self-organizing net also has the capability to form a representative pattern for each of the given sets of pattern stimuli. All of these capabilities are studied theoretically.

Pattern ξ



The Amari-Hopfield Model

Pattern ξ



Q: 如何采用Hebb学习规则将一个image (记为pattern ξ) 存储于网络中?

前提: 也把该image视为神经元活动状态的pattern

- 把pattern ξ 等分成 $N = 28 * 28$ 的pixels
- 采用 N 个neuron来分别表示pattern ξ 中的所有pixels
- 定义每个neuron的状态 x_i ($i = 1, \dots, N$) 为相应pixel上的数值, 令 $x_i = \pm 1$ (例如: 绿色的pixel为1, 蓝色底板的pixel为-1)
- 根据Hebb学习规则, 则任意neuron i 和neuron j 之间的突触连接强度为: $w_{ij} = w_{ji} = x_i x_j$ ($i \neq j$) 且 $w_{ii} = 0$; $|w_{ij}| \leq 1$

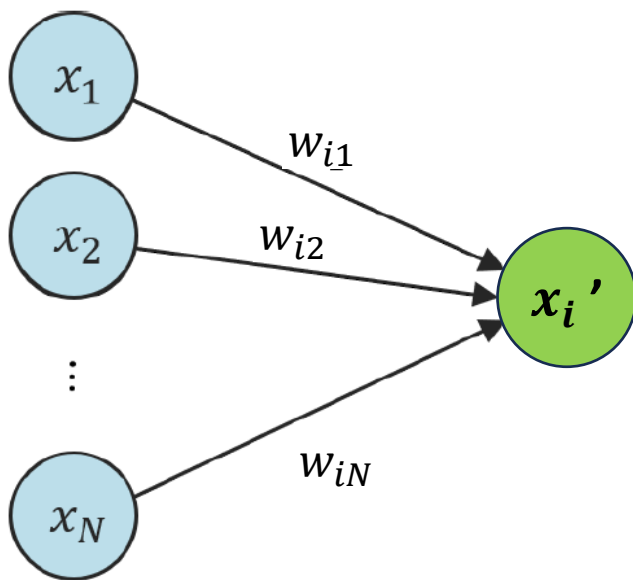
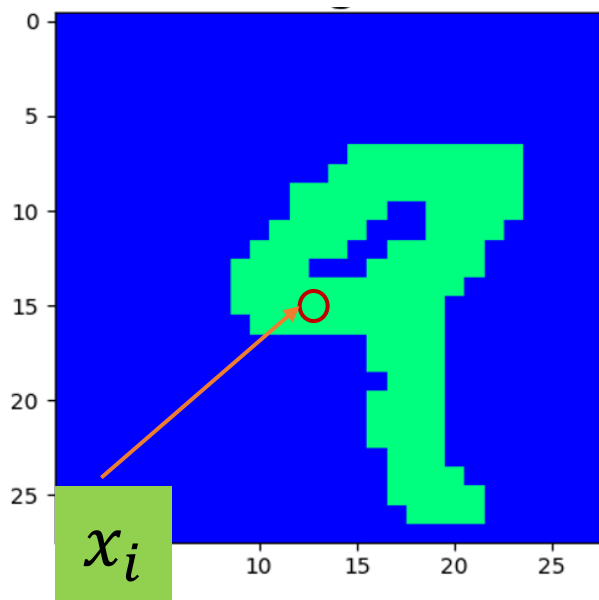
备注: 神经元 j 指向神经元 i 的突触连接记为 w_{ij} ; 所有蓝色pixel的神经元之间突触连接强度为1; 所有绿色pixel的神经元之间的突触连接强度为1; 任意绿色和蓝色pixel之间的突触连接强度为-1;

事实上, 通过Hebb学习规则, 我们构建了一个由 N 个神经元构成的网络模型, 任意神经元 i & j 之间突触连接的强度则为 w_{ij} 。网络一旦通过Hebb学习规则被训练后, 网络的结构将被固定, 不再发生变化。

The Amari-Hopfield Model

Q: 存储于网络中的pattern ξ 是否稳定?

Pattern ξ



Binary neurons with discrete time:

在构建的Amari-Hopfield Model中，任意神经元 $j(j = 1, \dots, N)$ 与神经元 i 连接，且连接强度为 w_{ij} 。也就是说，任意 t 时刻，神经元 i 收到的来自第 j 个神经元的信号为 x_j ，且该信号的强度为 w_{ij} 。

通常假定神经元 i 对所有输入信号的响应 x_i' 为所有输入信号的线性叠加，即

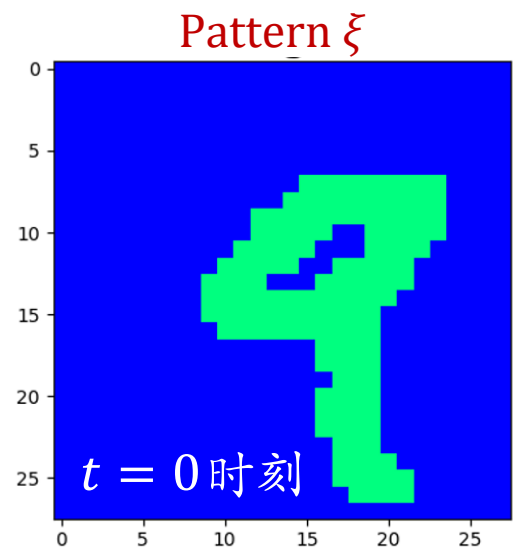
$$x_i' = \sum_{j=1}^N w_{ij} x_j - \theta$$

网络中任意神经元 i 的活动状态的更新规则:

If $x_i' = \sum_j w_{ij} x_j - \theta \geq 0$, $x_i' \rightarrow +1$

If $x_i' = \sum_j w_{ij} x_j - \theta < 0$, $x_i' \rightarrow -1$

The Amari-Hopfield Model



网络中任意神经元*i*的活动状态的更新规则:

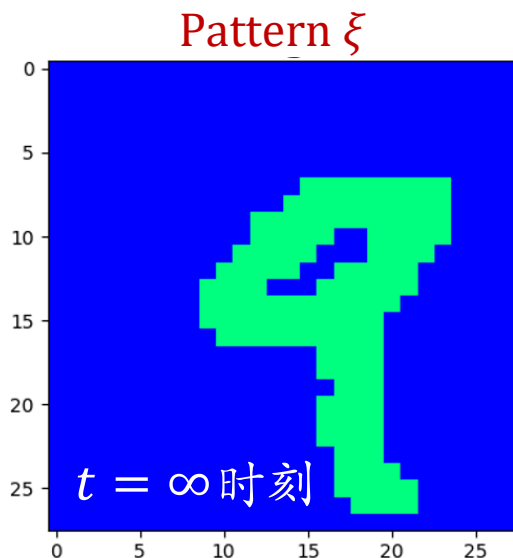
If $\sum_j w_{ij}x_j(t) - \theta \geq 0$, $x_i(t+1) \rightarrow +1$

If $\sum_j w_{ij}x_j(t) - \theta < 0$, $x_i(t+1) \rightarrow -1$

$$x_i(t+1) = \text{sign}\left[\sum_j w_{ij}x_j(t) - \theta\right]$$

Stability & Equilibrium State:

$$\mathbf{X}(t+1) = \mathbf{TX}(t)$$

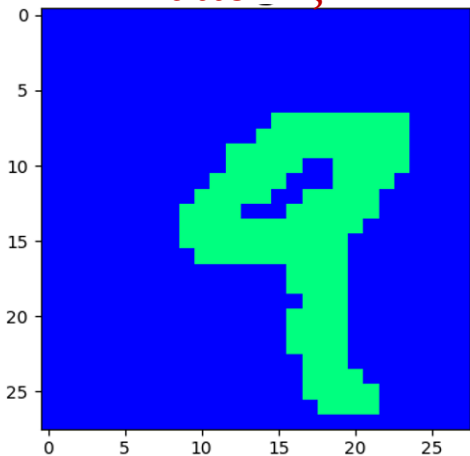


Updates in the Hopfield network can be performed in two different ways:

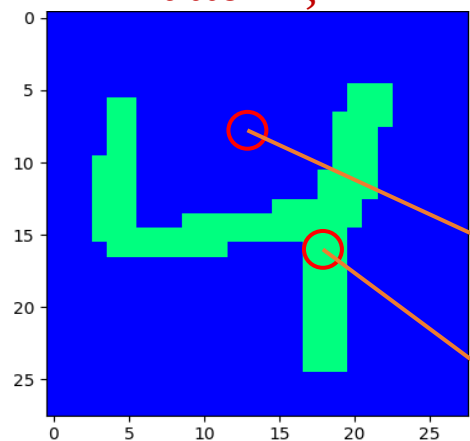
- **Asynchronous:** Only one unit is updated at a time. This unit can be picked at random, or a pre-defined order can be imposed from the very beginning.
- **Synchronous:** All units are updated at the same time. This requires a central clock to the system in order to maintain synchronization. This method is viewed by some as less realistic, based on an absence of observed global clock influencing analogous biological or physical systems of interest.

The Amari-Hopfield Model

Pattern ξ^1



Pattern ξ^2



Q: 如何采用Hebb学习规则将多个pattern (如 ξ^1 & ξ^2) 存储于网络中?

意味着: 网络中所有神经元集体响应的活动pattern既可以呈现出9, 也可以呈现出4

- 把pattern ξ^1 & ξ^2 等分成 $N = 28 * 28$ 的pixels
- 采用 N 个neuron来分别表示pattern ξ^1 & ξ^2 中的所有pixels
- 定义每个neuron的状态 x_i^k ($i = 1, \dots, N, k = 1, K; K = 2$) 为相应pixel上的数值, 令 $x_i^k = \pm 1$
- 根据Hebb学习规则, 则任意neuron i 和neuron j 之间的突触连接强度为

$$w_{ij} = \frac{1}{K} \sum_{k=1}^K x_i^k x_j^k$$

- 网络一旦通过Hebb学习规则被训练后, 如上两个pattern便可存储在网络中了。

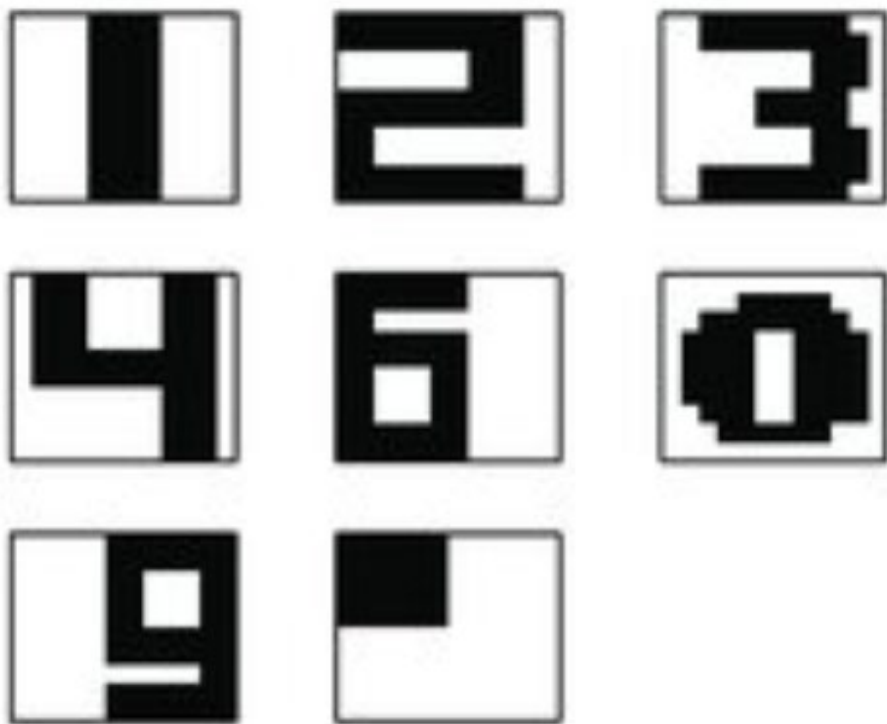
$$x_j^2 = -1$$

$$x_i^2 = +1$$

网络中任意神经元 i 的活动状态的更新规则:

- If $\sum_j w_{ij} x_j(t) \geq \theta$, $x_i(t+1) \rightarrow +1$
- If $\sum_j w_{ij} x_j(t) < \theta$, $x_i(t+1) \rightarrow -1$

The Amari-Hopfield Model



$$w_{ij} = \frac{1}{K} \sum_{k=1}^K x_i^k x_j^k$$

Memory Capacity (K_c) in case of random memories :

$$K_c \sim 0.14N$$

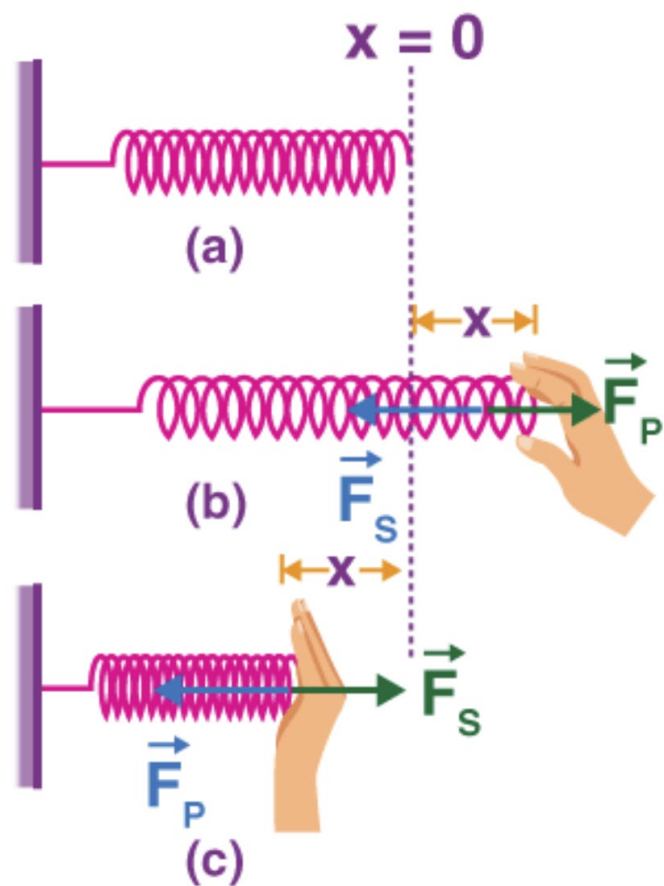
Amari-Hopfield model采用了将多个pattern（记忆，associative memory）存储于神经元之间突触连接上的观点。

神经突触：传递信息；存储记忆（长时记忆）

大脑中有100亿个神经元，按照Amari-Hopfield model中记忆容量的推测，我们则可记忆多少长时记忆信息？

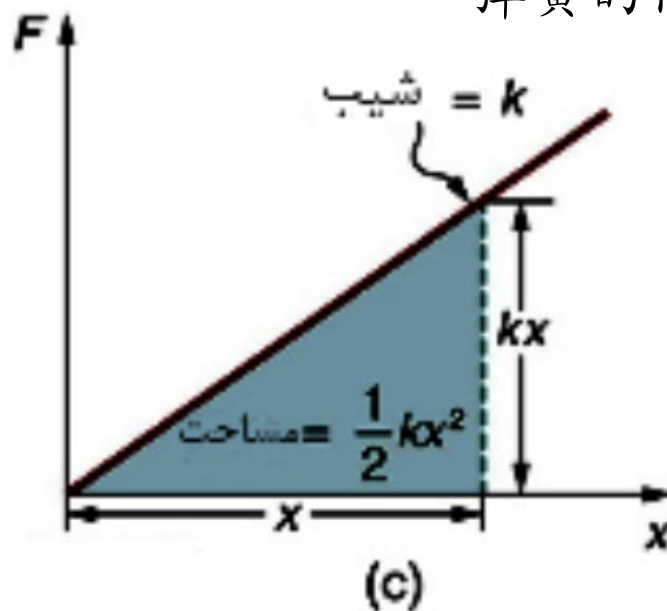
海量的长时记忆信息！！！！

The Energy Function of Amari-Hopfield Model



举例说明一下什么是能量函数。

e.g. 弹簧的能量函数: $E = \frac{1}{2}kx^2$, 其中, x 是弹簧的伸长量或压缩量, k 是弹簧的常数

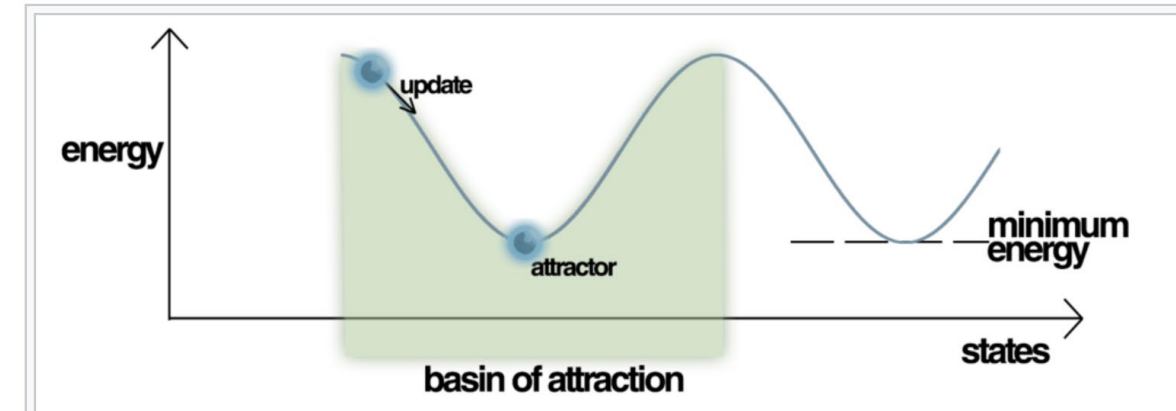


能量函数可以用来描述整个系统的状态, 通过最小化或最大化这个能量函数, 我们可以找到系统最合适的状态

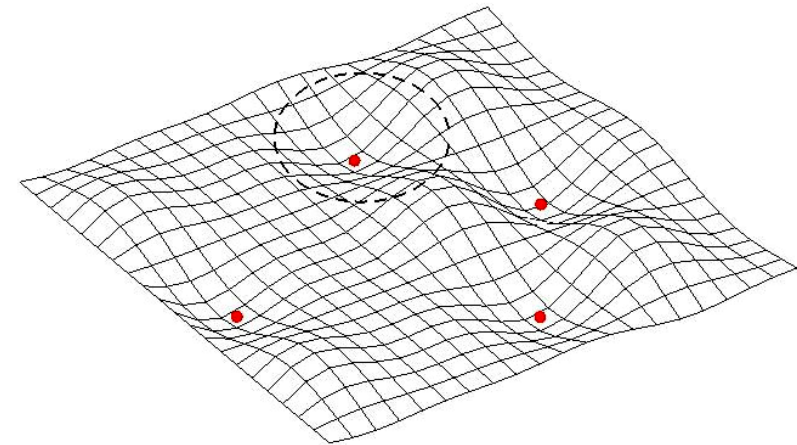
The Amari-Hopfield Model

$$\text{Update Rule : } x_i(t + 1) = \text{sign}\left[\sum_j w_{ij}x_j(t) - \theta\right]$$

$$\text{Energy Function : } E = -\frac{1}{2}\sum_{i,j} w_{ij}x_ix_j + \theta \sum_i x_i$$



$$\begin{aligned}\Delta E &= E(t + 1) - E(t) \\ &= -[x_i(t + 1) - x_i(t)] \sum_j w_{ij}x_j(t) + \theta [x_i(t + 1) - x_i(t)] \\ &= -[x_i(t + 1) - x_i(t)] \left[\sum_j w_{ij}x_j(t) - \theta \right] \\ &\leq 0\end{aligned}$$



Under repeated updating, the network will eventually converge to a state which is a local minimum in the energy function. Thus, if a state is a local minimum in the energy function it is a stable state for the network.

Neural Network Computing

Neural network computation or analog computation can similarly be described as **state space flows** but without the discretization of state variables and time.

A computation is begun by setting the computer in an initial state determined by standard initialization + program + data. At each tick of the computer clock the state changes into another state, following a rule that is built in by the design of the machine. After a while the computer stops changing its state, and the answer to the computation is read out from a subset of the storage registers

For general neural networks, the only way to understand what future state will evolve after a long time is by following the dynamics in detail.

The Amari-Hopfield Model

Hopfield networks have a Lyapunov function E (or ‘energy function’) behind their dynamics which leads to an understanding of possible final states.

The Lyapunov function decreases in a monotone fashion under the dynamics, and is bounded below.

Computation in this system can now be thought of as being done by starting from an initial state from which a final state evolves by moving downhill on E .

Because of the existence of an elementary Lyapunov function for the dynamics, the only possible asymptotic result is a state on **an attractor**.

Hopfield networks have been shown to be capable of universal computation in the Turing sense.

Associative memory

Associative Memory (AM) is a core concept in psychology responsible for linking related items.

The functionality of Associative Memory is modeled by an energy function, where low values of the energy correspond to stored memories and constitute the most likely states of the system.

Powerful error correction mechanisms that are constantly working in your brain that associate imperfect inputs with the proper semantic meaning of individual words

Associative Memory is a content addressable information storage system that is capable of error correction

Association

Connect inputs to impose structure on a complex world



Shape+color with universal meaning



Name that movie!



What does this picture "smell" like?

Memory

Leverage association to recall missing information



Who is this?



What color is her hair?

Error Correction

Filter corruption to detect meaning behind the noise

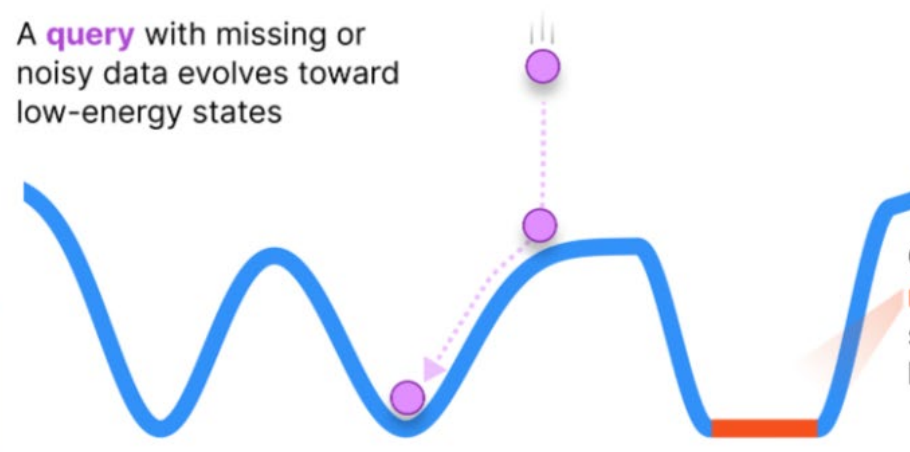
According to a research study at Cambridge University, it doesn't matter in what order the letters in a word are, the only important thing is that the first and last letter be at the right place. The rest can be a total mess and you can still read it without problem.

Associative Memory

Unifies these three ideas via **energy** minimization



A **query** with missing or noisy data evolves toward low-energy states



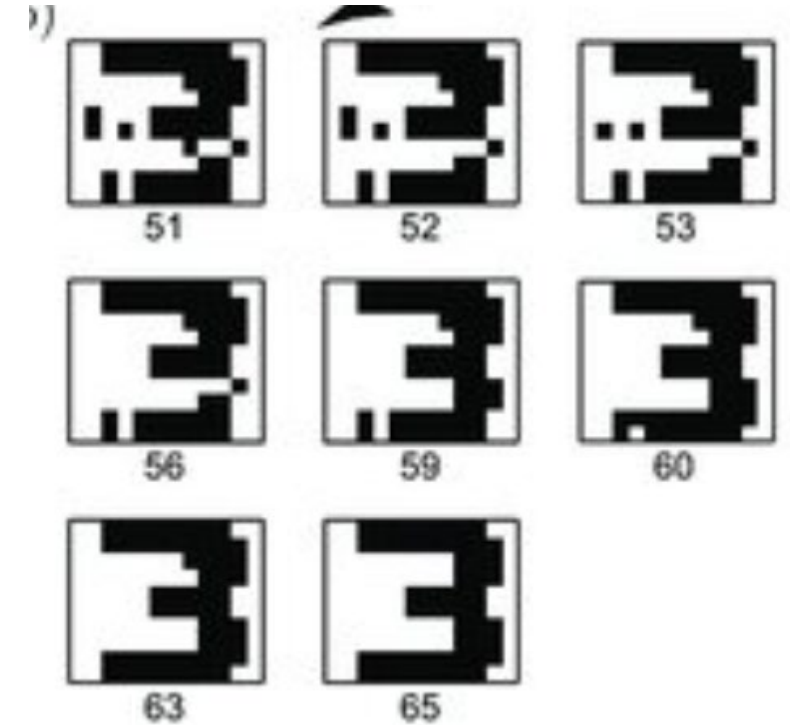
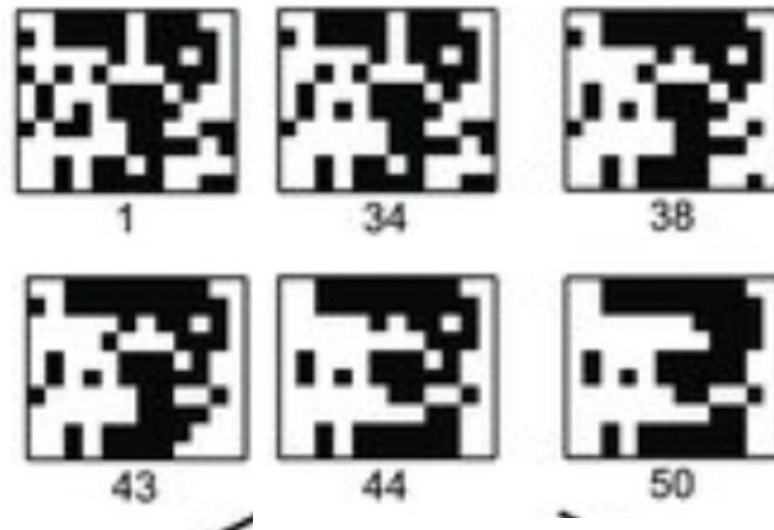
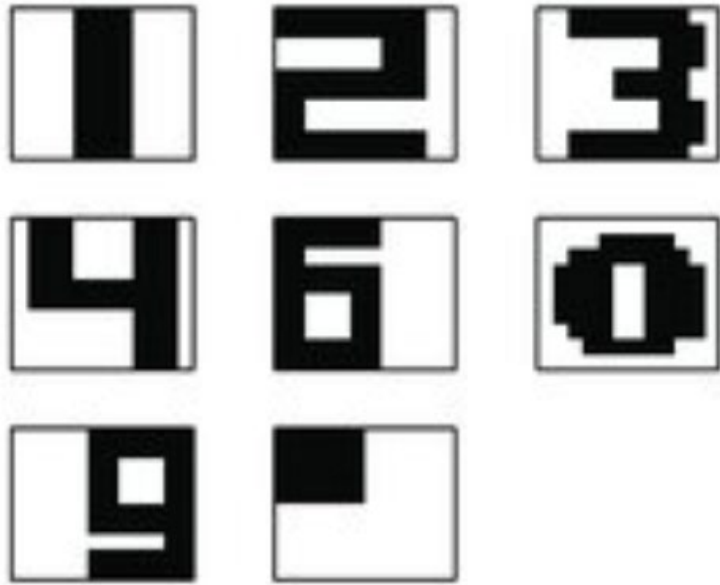
Can also converge to **manifolds** where many solutions have equally low energy

Associative memory

Consider a physical system described by many coordinates $X_1 \cdots X_N$, the components of a state vector X . Let the system have locally stable limit points $X_a, X_b, \cdots$. Then, if the system is started sufficiently near any X_a , as at $X = X_a + \Delta$, it will proceed in time until $X \approx X_a$. We can regard the information stored in the system as the vectors $X_a, X_b, \cdots$. The starting point $X = X_a + \Delta$ represents a partial knowledge of the item X_a , and the system then generates the total information X_a .

Any physical system whose dynamics in phase space is dominated by a substantial number of locally stable states to which it is attracted can therefore be regarded as a general content-addressable memory. The physical system will be a potentially useful memory if, in addition, any prescribed set of states can readily be made the stable states of the system.

Memory Retrieval



$$w_{ij} = \frac{1}{K} \sum_{k=1}^K x_i^k x_j^k$$

$$x_i(t+1) = \text{sign} \left[\sum_j w_{ij} x_j(t) - \theta \right]$$

Retrieve a stored memory pattern by partial or distorted presented image

The Amari-Hopfield Model 计算仿真实战

冯至韬

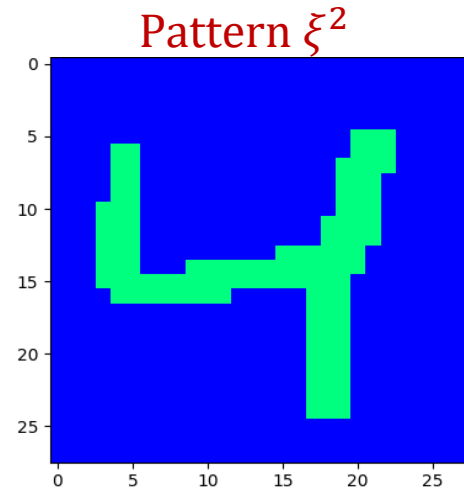
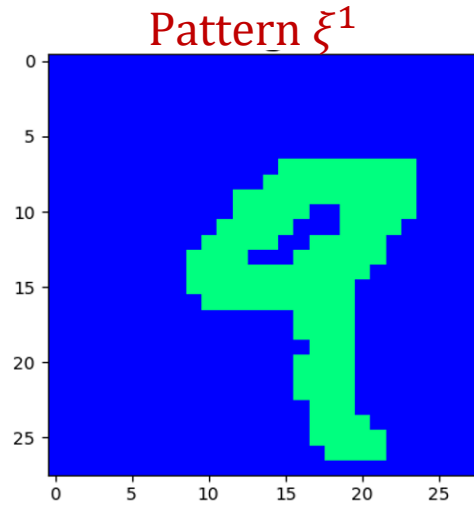
计算仿真的内容

1. 根据 Amari 1972 文章中的权重更新方法，习得可稳定存储多个 pattern（包括 sequential patterns）的网络连接权重

备注：pattern 包括 images 等

2. Memory Patterns 存储的稳定性，及其能量函数
3. Memory Retrieval 过程

1. 如何采用 Hebb 学习规则将多个 patterns 存储在 Amari-Hopfield Model 中?



- 把 pattern ξ^1 & ξ^2 等分成 $N = 28 * 28$ 的 pixels
- 采用 N 个 neuron 来分别表示 pattern ξ^1 & ξ^2 中的所有 pixels
- 定义每个 neuron 的状态 x_i^k ($i = 1, \dots, N, k = 1, K; K = 2$) 为相应 pixel 上的数值, 令 $x_i^k = \pm 1$
- 根据 Hebb 学习规则, 则任意 neuron i 和 neuron j 之间的突触连接强度为 $w_{ij} = \frac{1}{K} \sum_{k=1}^K x_i^k x_j^k$

$$W = x \cdot x^T = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \cdot [x_1 \quad x_2 \quad \cdots \quad x_n]$$
$$= \begin{bmatrix} x_1^2 & x_1 x_2 & \cdots & x_1 x_n \\ x_2 x_1 & x_2^2 & \cdots & x_2 x_n \\ \vdots & \vdots & \ddots & \vdots \\ x_n x_1 & x_n x_2 & \cdots & x_n^2 \end{bmatrix}$$

考虑到神经元自身对自身没有这样的突触连接:

$$W = xx^T - I = \begin{bmatrix} 0 & x_1 x_2 & \cdots & x_1 x_n \\ x_2 x_1 & 0 & \cdots & x_2 x_n \\ \vdots & \vdots & \ddots & \vdots \\ x_n x_1 & x_n x_2 & \cdots & 0 \end{bmatrix}$$

$$x_i(t+1) = \text{sign} \left[\sum_j w_{ij} x_j(t) - \theta \right]$$

加载目标信息图片的数据，并存入 data 中：

```
1 data_dir = Path(__file__).resolve().parent / "test_data.npy"
2 data = (np.load(str(data_dir))).astype(np.float32)
3 image1 = data[0]
4 image2 = data[1]
5 same_of_pics = (image1.reshape(28, 28) + image2.reshape(28, 28)) / 2
6 same_of_pics = np.where(same_of_pics < 1, -1, same_of_pics)
```

Uploaded image 的数据形式是 784×1 的 vector，即 $\mathbf{x}(1:N)$

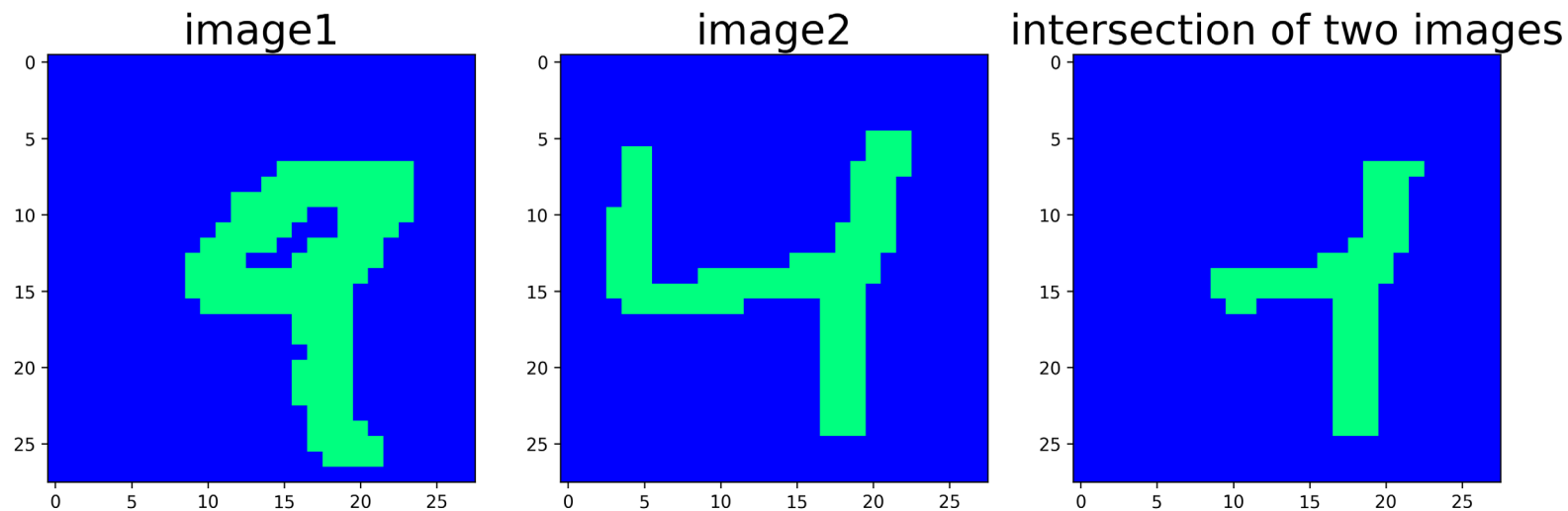
➤ 计算两张图片的重叠部分：

`Image1.reshape` 将图片的数据（ 784×1 的 vector）转为 28×28 的二维图像，两个图片的矩阵相加，除以 2，得到一个元素构成为 -1, 0, 1 的新矩阵，元素为 1 的位置表示对两张图片都产生发放活动的神经元位置。

用 `np.where` 找出没有发放活动的神经元位置，将这些位置的神经元响应都置为 -1。

可视化两张 image patterns:

```
1 fig, ax = plt.subplots(nrows=1, ncols=3, figsize=(15, 7.5))
2 ax[0].imshow(image1.reshape(28, 28), interpolation="None", cmap="winter")
3 ax[0].set_title("image1", fontsize=24)
4 ax[1].imshow(image2.reshape(28, 28), interpolation="None", cmap="winter")
5 ax[1].set_title("image2", fontsize=24)
6 ax[2].imshow(same_of_pics, interpolation="None", cmap="winter")
7 ax[2].set_title("intersection of two images", fontsize=24)
```



创建 AmariHopfieldNet 类，并初始化

根据 Hebb 学习规则构建网络

```
1 class AmariHopfieldNet(bp.DynamicalSystem):
2     def __init__(self, num):
3         super().__init__()
4         self.num = num
5         self.weight = bm.Variable(bm.zeros([num, num]))
```

构建存储信息的函数

```
1 @bm.cls_jit                                # JIT 编译加速
2 def store_patterns(self, samples):          # samples: 需要存储的图片信息数据，即图片的 memory pattern
3     assert samples.ndim == 2
4     assert samples.shape[1] == self.num
5
6     bm.for_loop(self.store, samples)        # 循环 2 次，每次存储一张图片的信息
7     bm.fill_diagonal(self.weight, 0)        # 神经元自身和自身没有连接，每次循环，将对角元素置 0
```

每次 for_loop 循环中，根据图片数据构建神经元连接矩阵 W

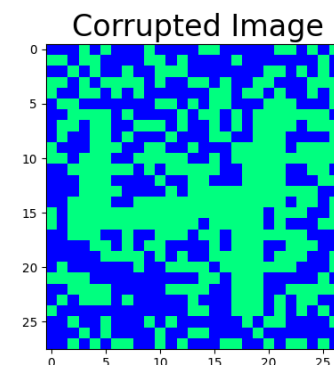
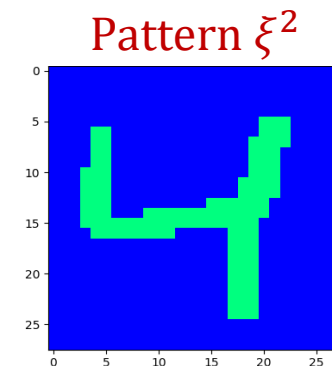
$$w_{ij} = \frac{1}{K} \sum_{k=1}^K x_i^k x_j^k \Rightarrow \mathbf{w} = \frac{1}{K} \sum_{k=1}^K \mathbf{x}^k \cdot (\mathbf{x}^k)^T$$

```
1 @bm.cls_jit
2 def store(self, sample):
3     # sample is an array with the shape of (N,)
4     assert self.num == sample.shape[0]
5
6     # Data outer-product gives neural hopfield update rule.
7     w_update = bm.outer(sample, sample)
8
9     # Sum all pattern outer-products.
10    self.weight += w_update
```

$$W = x \cdot x^T = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \cdot [x_1 \quad x_2 \quad \cdots \quad x_n]$$
$$= \begin{bmatrix} x_1^2 & x_1 x_2 & \cdots & x_1 x_n \\ x_2 x_1 & x_2^2 & \cdots & x_2 x_n \\ \vdots & \vdots & \ddots & \vdots \\ x_n x_1 & x_n x_2 & \cdots & x_n^2 \end{bmatrix}$$

根据异步方法更新神经活动 $\mathbf{x}(t)$

```
1 def async_recover(self, sample, n, energy=False): # sample: cue, 原始图片或加入 noise 的图片
2     # 需要迭代 n 次以更新输入 images 后网络所有神经元的响应, 故随机选择 n 个神经元进行更新
3     idxs = bm.random.randint(0, self.num, n)
4     sample = bm.Variable(sample)
5
6     def recover(i):
7         # 每次迭代, 某一个神经元根据来自其他神经元的输入更新自己的响应
8         sample[i] = bm.sign(bm.inner(self.weight[i], sample))
9         if energy:
10             # 每次迭代后计算网络的能量
11             return self.energy(sample)
12
13     r = bm.for_loop(recover, idxs) # for loop JIT
14     return (sample, r) if energy else sample
```



$$x'_i = \text{sign} \left[\sum_j w_{ij} x_j - \theta \right]$$

$= +1 \text{ or } -1$

构建能量函数

```
1 def energy(self, x):  
2     return bm.inner(-x @ self.weight, x)
```

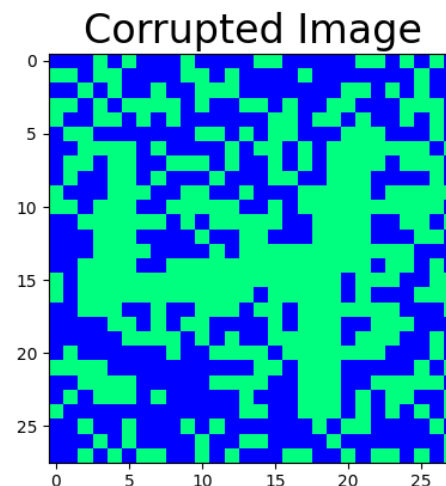
$$E = -\frac{1}{2}x^T W x$$

实例化 Amari-Hopfield model, 并将图片数据存储到网络中

```
1 net = AmariHopfieldNet(num=data.shape[1])  
2 net.store_patterns(np.vstack((image1, image2)))
```

给 image2 增加 bernouli_noise, 为了展示 memory retrieval

```
corrupted_image2 = add_bernouli_noise(image2, prob=0.4)
```

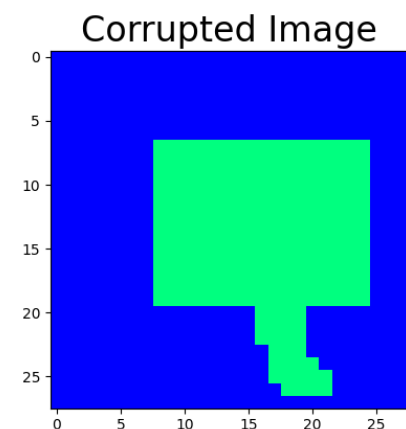
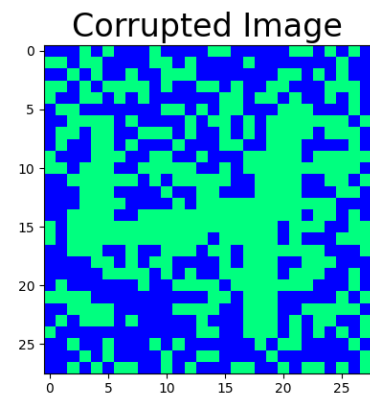


调用如下函数便可得到 retrieved memory pattern 2 以及 retrieval process 中网络能量函数随时间的变化

```
1 n_iterations = 10000  
2 cleaned_image, energy_vec = net.async_recover(corrupted_image2, n_iterations, True)
```

定义两种向图片加入噪声的形式

```
1 def add_bernouli_noise(img, prob=0.4, mask_value=1.0):
2     # mask_value → 0: original; 1: corrupted
3
4     rand = np.random.random(img.shape) < prob
5     return np.where(rand, mask_value, img)
6
7
8 def add_mask_noise(img, mask_value=-1, mask_matrix=np.zeros((2, 2))):
9     img = np.copy(img)
10
11     # 取 mask_matrix 第一行元素作为覆盖原图片矩阵数据的行范围
12     # 取 mask_matrix 第二行元素作为覆盖原图片矩阵数据的列范围
13     img[
14         mask_matrix[0, 0] : mask_matrix[0, 1], mask_matrix[1, 0] : mask_matrix[1, 1]
15     ] = mask_value
16     return img
```



add_bernouli_noise:

取图片数据中的一些随机位置，增加一些无关的信息：取与图片数据数组大小相同的 (0, 1) 随机数组，并在随机数组中找到小于 prob 的位置，prob 决定噪声大小。

add_mask_noise:

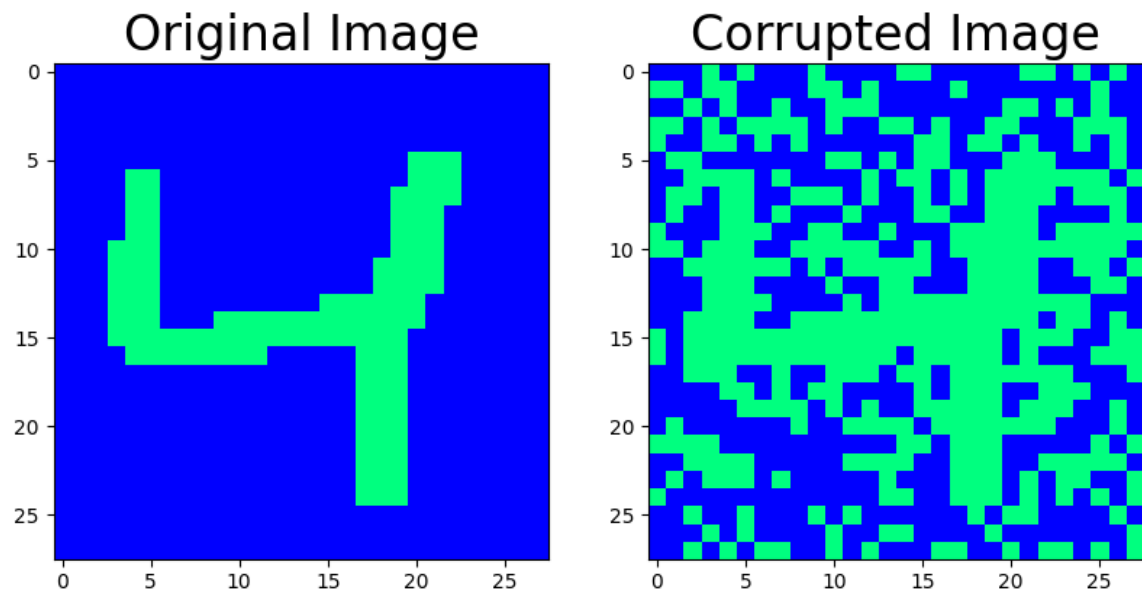
取图片 28*28 矩阵数据中的随机一块区域，用 -1 或 1 覆盖：取一个 2*2 矩阵 mask_matrix，矩阵第一行的两个值决定图片矩阵数据中被覆盖区域的行范围，mask_matrix 第二行的值决定被覆盖区域的列范围。

分别加入两种噪声后的图像

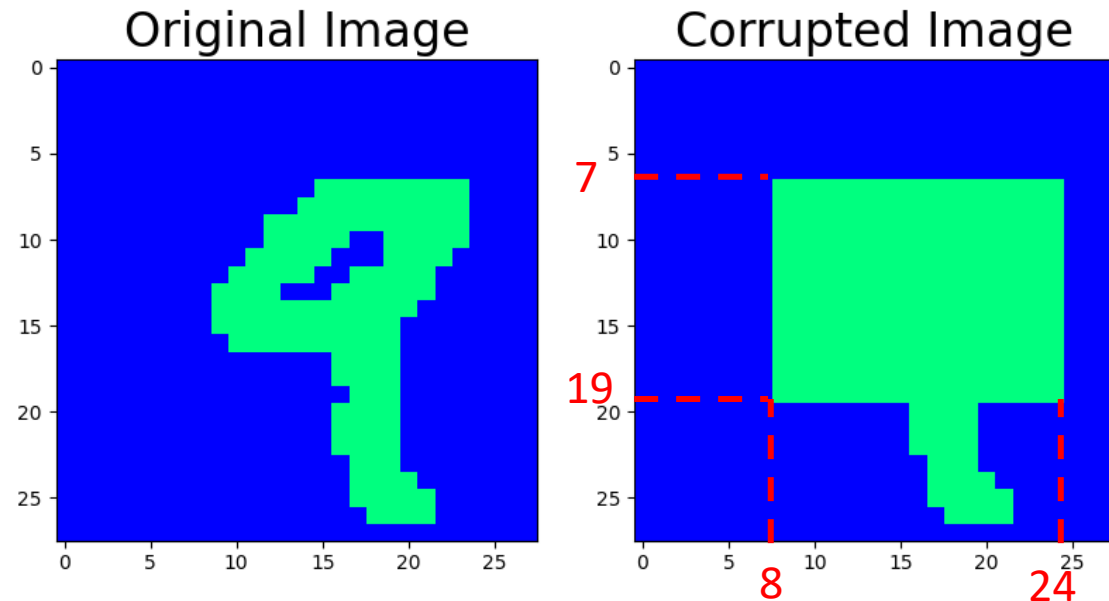
```
corrupted_image2 = add_bernouli_noise(image2, prob=0.4)
```

```
1 mask_matrix = np.asarray([[7, 20], [8, 25]], dtype=np.int32)
2 corrupted_image1 = add_mask_noise(
3     image1.reshape(28, 28), mask_value=1, mask_matrix=mask_matrix
4 )
```

add_bernouli_noise



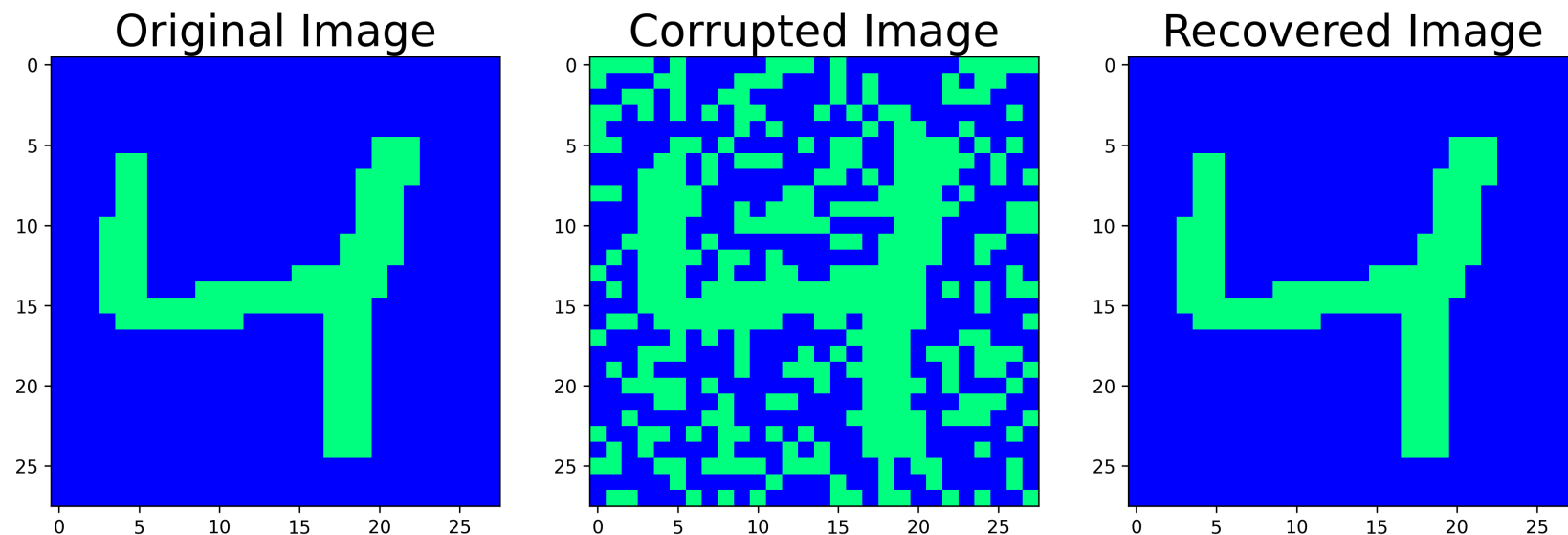
add_mask_noise



画出原始图片，cue 以及网络根据 cue 回忆出的图片 2

Memory Retrieval

```
1 fig, ax = plt.subplots(nrows=1, ncols=3, figsize=(15, 7.5))
2 ax[0].imshow(image2.reshape(28, 28), interpolation="None", cmap="winter")
3 ax[0].set_title("Original Image", fontsize=24)
4 ax[1].imshow(corrupted_image2.reshape(28, 28), interpolation="None", cmap="winter")
5 ax[1].set_title("Corrupted Image", fontsize=24)
6 ax[2].imshow(cleaned_image.reshape(28, 28), interpolation="None", cmap="winter")
7 ax[2].set_title("Recovered Image", fontsize=24)
```

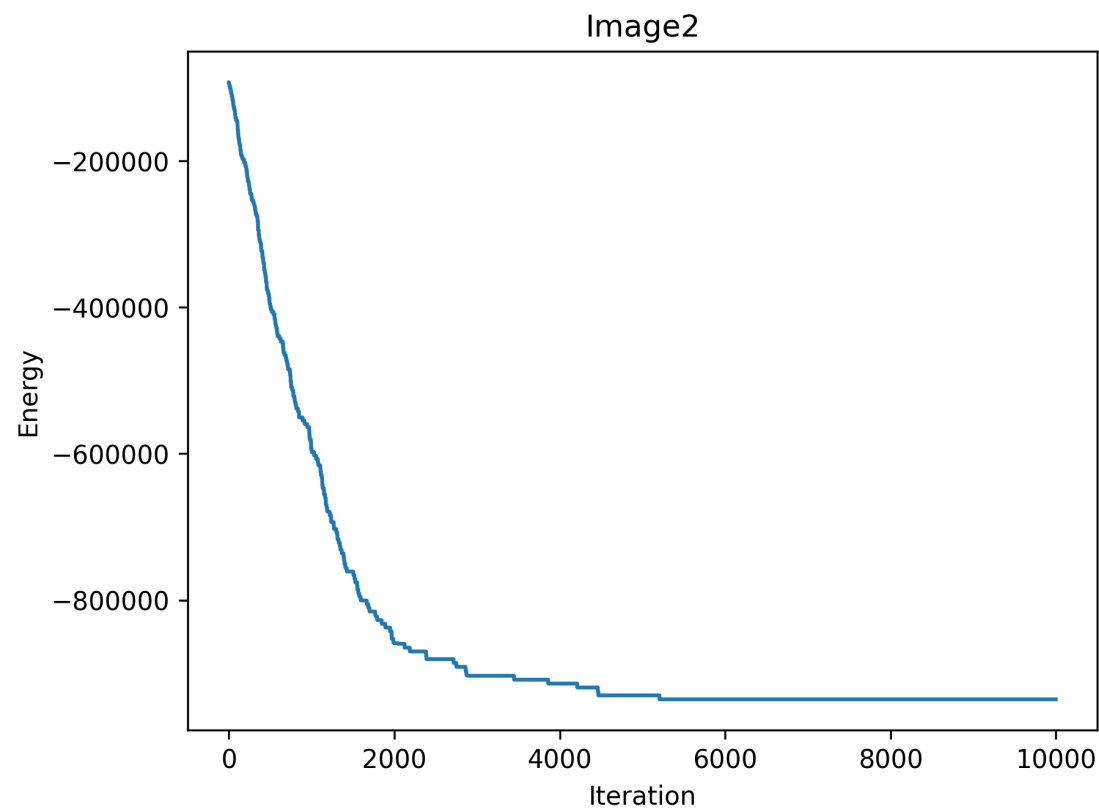


尝试改变不同强度的噪声，包括 $p=0$ 时；

探索不同强度噪声下能量函数随时间演化有何不同？

画出网络回忆 image2 过程中能量函数的变化，能量函数达到了极小值

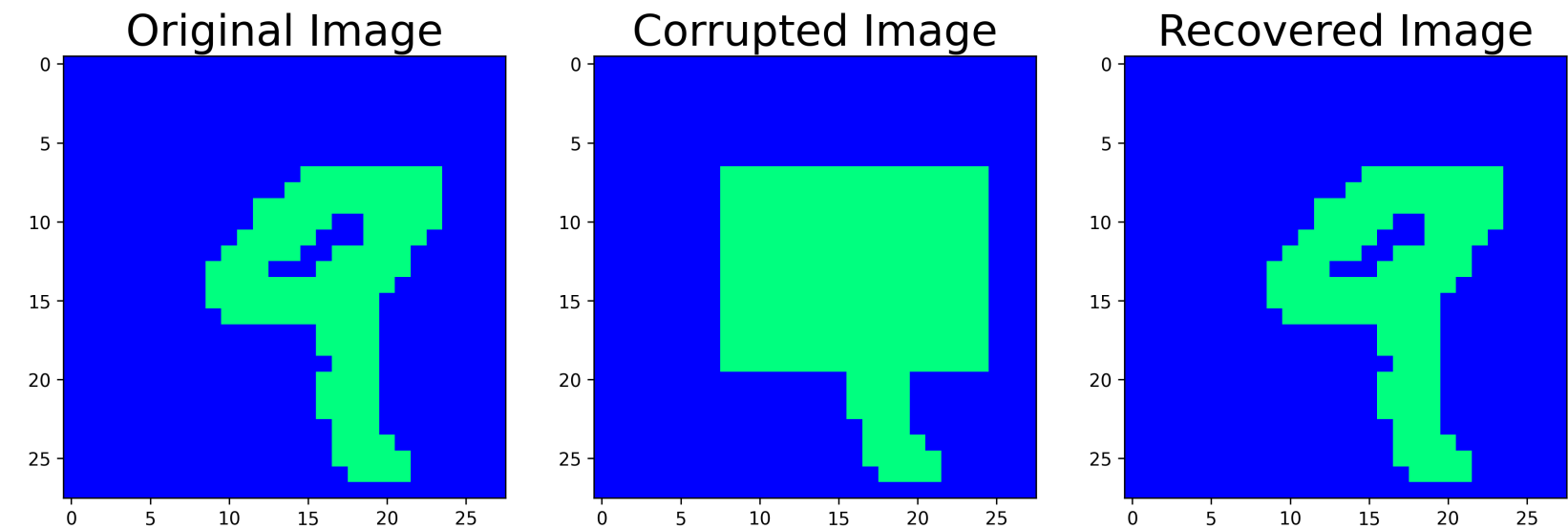
```
1 plt.figure()
2 plt.plot(bm.as_numpy(energy_vec))
3 plt.xlabel("Iteration")
4 plt.ylabel("Energy")
5 plt.title("Image2")
```



向 image1 加入另一种另一种 noise，作为 image1 的 cue，并根据 cue 得到网络回忆出的 image1

```
1 mask_matrix = np.asarray([[7, 20], [8, 25]], dtype=np.int32)
2 corrupted_image1 = add_mask_noise(
3     image1.reshape(28, 28), mask_value=1, mask_matrix=mask_matrix
4 )
```

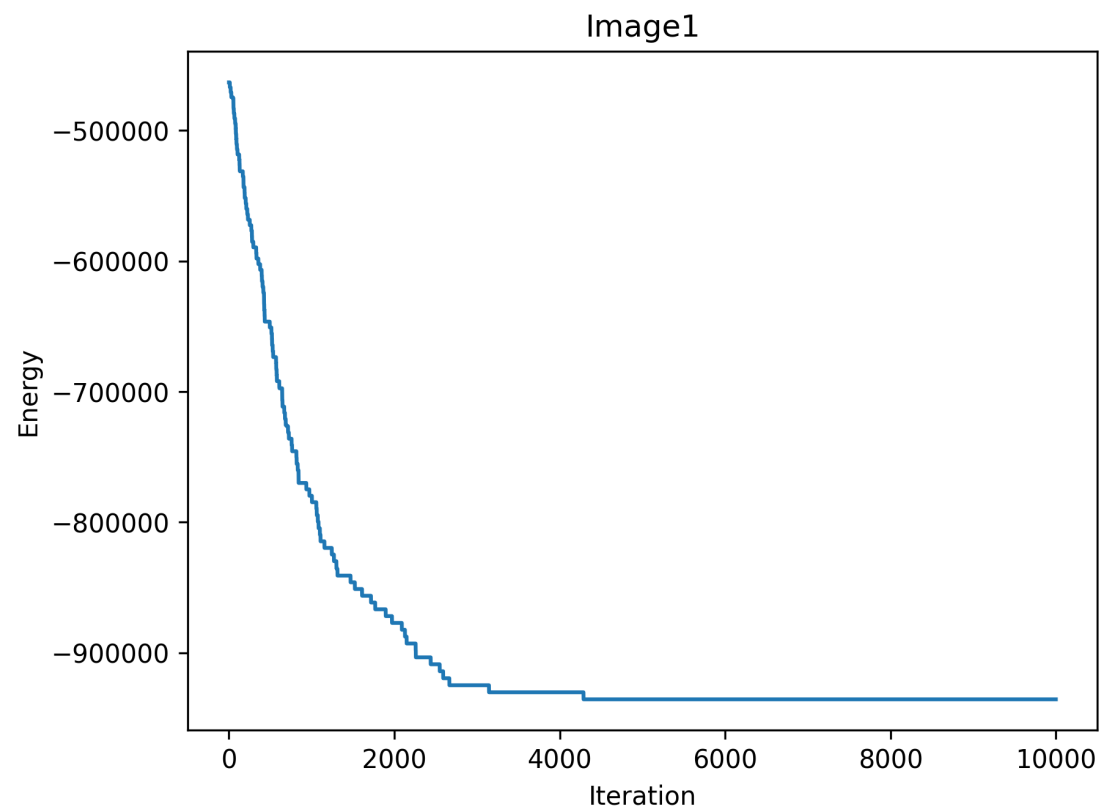
```
1 n_iterations = 10000
2 cleaned_image, energy_vec = net.async_recover(
3     corrupted_image1.flatten(), n_iterations, True
4 )
```



尝试改变 mask 的大小和位置
探索不同 mask 下能量函数随时间演化有何不同?
探索成功 retrieval memory pattern 的关键特征?

画出网络回忆 image1 过程中能量函数的变化，能量函数达到了极小值

```
1 plt.figure()  
2 plt.plot(bm.as_numpy(energy_vec))  
3 plt.xlabel("Iteration")  
4 plt.ylabel("Energy")  
5 plt.title("Image1")
```



Memory Retrieval

你看见哪两个字了呢？

土 虎

Memory Retrieval

intersection of two images

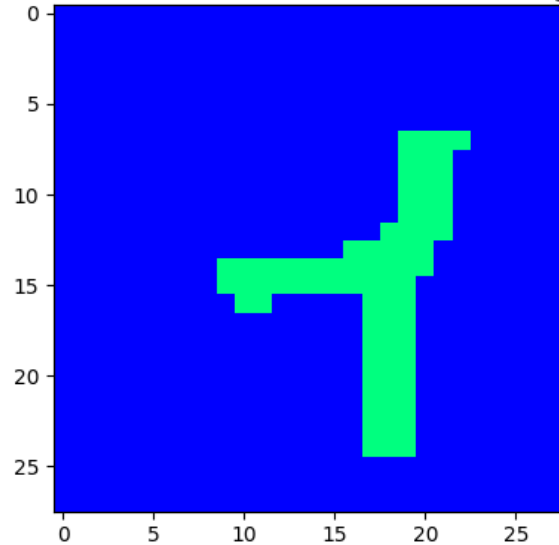


image1

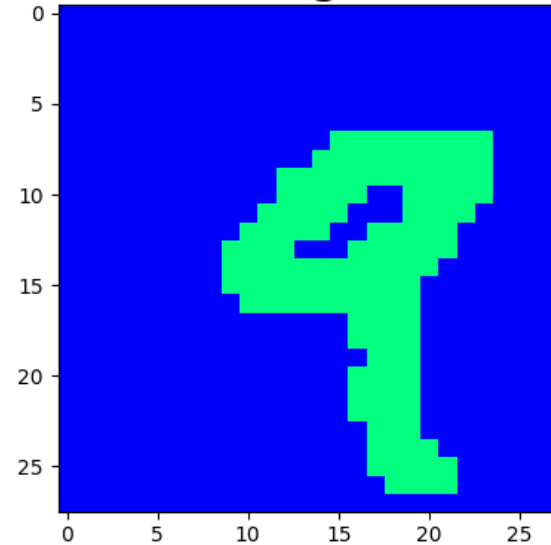
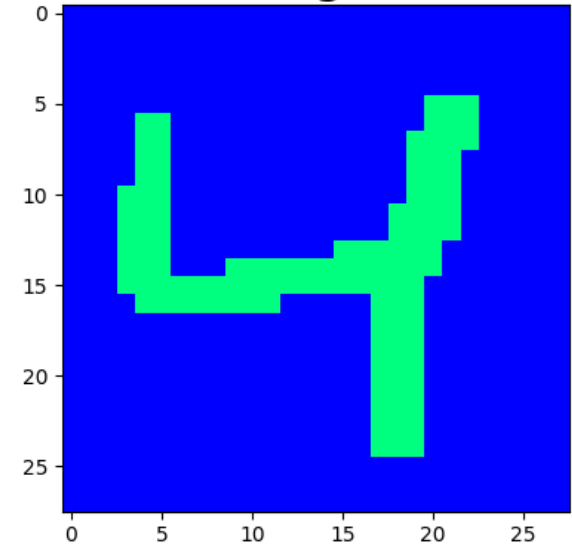


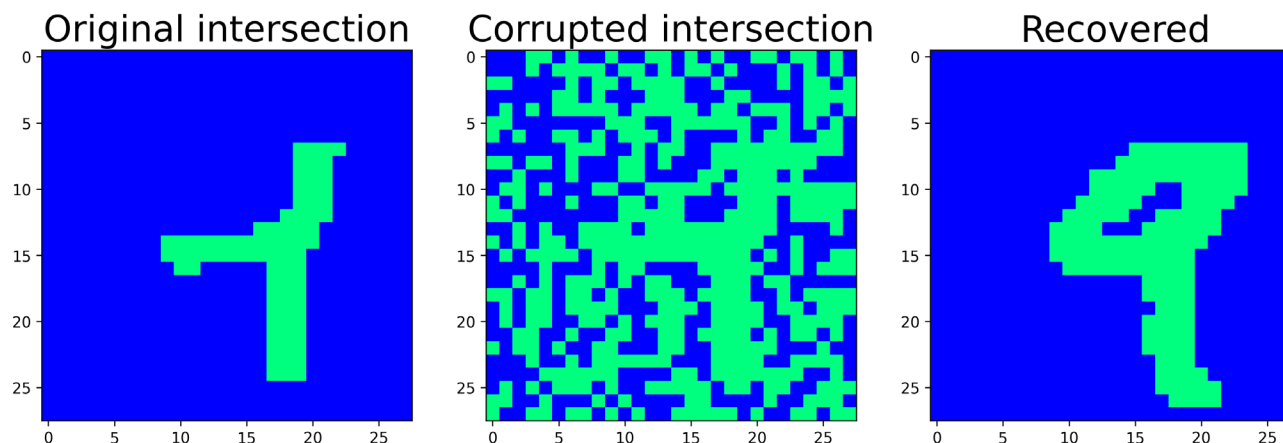
image2



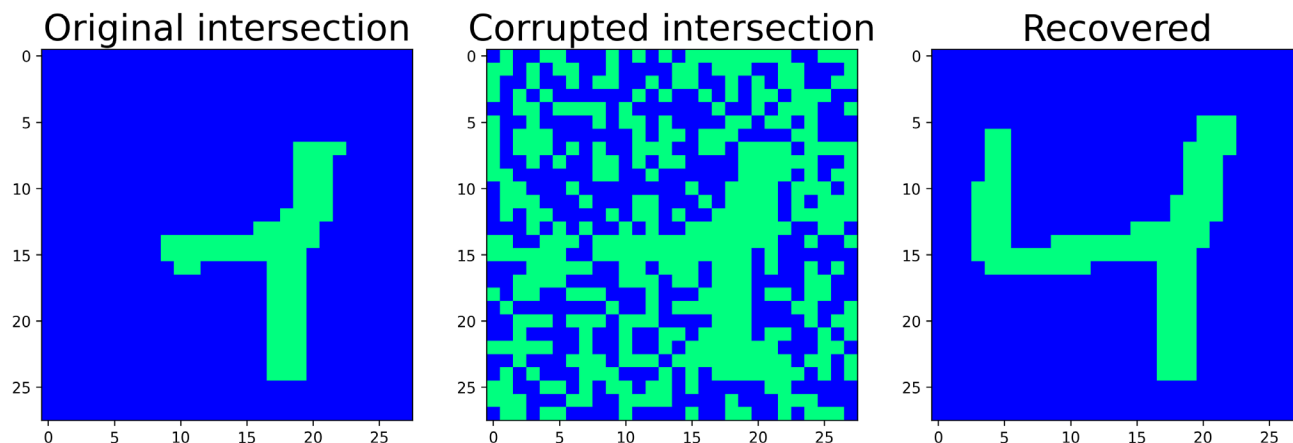
输入关于两张图片相交部分的 cue，并加入 noise，网络可能回忆出任意一张图片

```
1 corrupted_intersection = add_bernouli_noise(same_of_pics, prob=0.5)
2 recovered_interseciton, energy_vec = net.async_recover(
3     corrupted_intersection.flatten(), n_iterations, True
4 )
5 fig, ax = plt.subplots(nrows=1, ncols=3, figsize=(15, 7.5))
6 ax[0].imshow(same_of_pics, interpolation="None", cmap="winter")
7 ax[0].set_title("Original intersection", fontsize=24)
8 ax[1].imshow(corrupted_intersection, interpolation="None", cmap="winter")
9 ax[1].set_title("Corrupted intersection", fontsize=24)
10 ax[2].imshow(
11     recovered_interseciton.reshape(28, 28), interpolation="None", cmap="winter"
12 )
13 ax[2].set_title("Recovered", fontsize=24)
14 plt.show()
```

运行第一次，网络回忆出 image1:



运行第二次，网络回忆出 image2:



- 尝试修改 noise 类型和强度，运行多次查看网络的回忆结果。
- 尝试使用其他图片（文件 other_images.npy，即另外 5 张图片）数据运行代码。

改变噪声的强度（会有一个较好的强度范围），尝试噪声强度对 retrieval 之间的切换有何影响
同时探索一下，如上 memory retrieval 过程中能量函数的改变

Modern Hopfield Model:

Krotov D, Hopfield J J. Dense associative memory for pattern recognition[J]. Advances in neural information processing systems, 2016, 29.

Ramsauer H, Schäfl B, Lehner J, et al. Hopfield networks is all you need[J]. arXiv preprint arXiv:2008.02217, 2020.

Krotov D, Hopfield J. Large associative memory problem in neurobiology and machine learning[J]. arXiv preprint arXiv:2008.06996, 2020.

Demircigil M, Heusel J, Löwe M, et al. On a model of associative memory with huge storage capacity[J]. Journal of Statistical Physics, 2017, 168(2): 288-299.

Krotov D, Hopfield J. Dense associative memory is robust to adversarial inputs[J]. Neural computation, 2018, 30(12): 3151-3167.

Krotov D. Hierarchical associative memory[J]. arXiv preprint arXiv:2107.06446, 2021.

Its Applications of Modern Hopfield Model:

Kozachkov L, Kastanenko K V, Krotov D. Building transformers from neurons and astrocytes[J]. Proceedings of the National Academy of Sciences, 2023, 120(34): e2219150120.